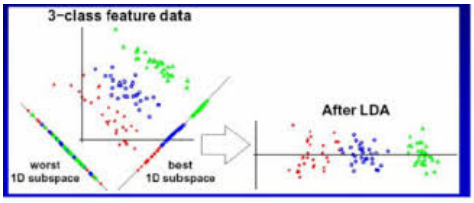
Computational Biology

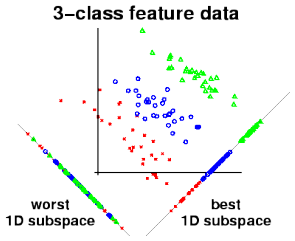
Lecture 10: *Exploring Biological Data*



I am not giving this lecture. Classification, Regression, Decision Trees, LDA, Feature Selection

Outline

1. **Classification**
2. Finding Relevant Variables
3. Feature Selection
4. Clustering



Data Exploration

- Machine learning is often used to make predictions
- But it can also be used for understanding data
- You don't need to be a great expert in machine learning
  - Use existing tools (e.g. scikit-learn)
  - Use common sense!

Coursework

- As I am no longer running an assignment this year I am not giving this lecture
- I leave the lecture on the web pags as it does provide a high level introduction to understanding data which some people might find useful
- You will not be examined on any of this material

The Data

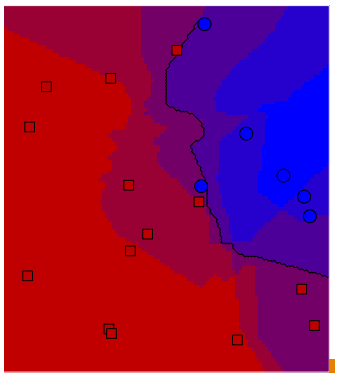
- You are being asked to look at some cancer data
- There are different data sets belonging to different cancers
- Real data is noisy, messy, incomplete, often rather sparse
- You are **not** being asked to come up with a good algorithm for determining the type of cancer
- You are being asked to explore the data and see what it tells you (if anything)

Classification

- One of the simplest analysis is to see if you can make a classifier that makes predictions
- There are lots of classifies that do this (use someone else's)
  - KNN
  - Perceptron, logistic regression, MLP
  - Decision trees, random forest, XGBoost
  - SVM, . . .

K-Nearest Neighbours

3-NN



Preparing Data

- Before running a learning machine you need to look at the data (pandas isn't bad for this)
- The data might be numeric or categorical
- Often you might have missing data
- Many techniques require numeric values (can you turn categorical data into numerical data?)
- Is your data normalised (are they over roughly same range?)
- Missing data can sometimes be replaced (e.g. with mean values)
- Decision trees can cope both with categorical data and missing values

Binary or Multi-Class

- There are lots of different classes—you don't need to look at all of them
- You could just try to identify one class, using all the others as examples of not that class (SVMs, perceptrons and logistic regression do binary classification)
- You can look at multi-class trying to classify the different classes (you might want to use KNN, a MLP with $softmax$ or decision tree based algorithms)

Balancing Datasets

- Often learning machines struggle doing classification when the datasets are very unbalanced
- In binary classification you might have many more positive examples than negative examples
- In multi-class classification you might have some classes with many more examples than others
- In this case it can sometimes help to balance the datasets either
  - By throwing away some data or
  - Using data of rare classes multiple times

Measuring Prediction Performance

- To see whether we can predict a cancer we must test our classifier on unseen data
- To do this we have to split the data into a **training** and **test** set
- We use the training set to train the data and the test set to test the algorithm
- There is a dilemma: when we have a small amount of data
  - We want to use as much data as possible for learning (to get good performance)
  - We want to use as much data as possible for testing (to get an accurate measure of performance)

Cross Validation

- We can solve our dilemma by repeating the training many times on different partitionings
- This is known as K-fold cross validation

$$\mathcal{D} = \{D_i\}_{i=1}^{P} \quad D_i = (\boldsymbol{x}_i, y_i)$$

$$E_{97} = \frac{5.5 + 3.5 + 1.8 + 3.8 + 3.7 + 3.6 + 7.4 + 4.6 + 0.99 + 4.5 + 4.6 + 5.4 + 6.2 + 3.3 + 2.7}{10} = 4.3$$

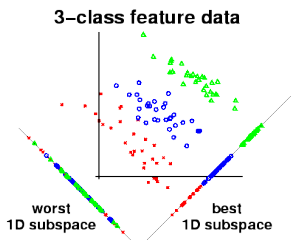
- Leave-one-out cross-validation is extreme case

Price of Cross Validation

- Cross-validation is computationally expensive (K-fold cross-validation requires K times as much work)
- There are attempts at estimating generalisation error more cheaply (boot-strapping) methods, but these are not very accurate
- Cross-validation is only necessary when you have little data
- Re-running code is usually cheap compared with writing the code

Outline

1. Classification
2. **Finding Relevant Variables**
3. Feature Selection
4. Clustering



What's Relevant

- Not all variables are necessarily relevant
- There are a couple of quick ways of seeing which variables are most relevant
  - Logistic Regression
  - Decision Trees

Logistic Regression

- In logistic regression we learn a function

$$f(\boldsymbol{x}) = \text{sigmoid}(\boldsymbol{w}^{\mathsf{T}}\boldsymbol{x} + b) \qquad \text{sigmoid}(z) = \frac{1}{1 + \mathrm{e}^{-z}}$$

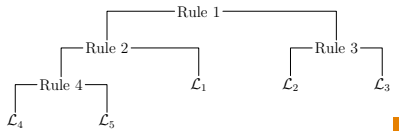
- The variables, x_i, with the largest (absolute) weights, $|w_i|$, are likely to be the most important (provided you have normalised your variables)
- This is a very standard algorithm that exists in most package, e.g. scikit-learn (in python)

Decision Trees

- Decision trees builds a binary tree to partition the data, $\mathcal{D} = \{(\boldsymbol{x}, y)\}$, into the leaves of the tree
- Each decision rule depends on a single feature
- At each step the rule is chosen that maximise the "*purity*" of the leaf nodes
- Decisions can be made on numerical values or categories

Partitioning

- Consider a classification problems with examples $(\boldsymbol{x}, y)$ belonging to some classes $y \in \mathcal{C}$
- The data is partitioned by the tree into leaves



- The proportion of data points in leaf $\mathcal{L}$ belonging to class c is

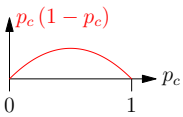
$$p_c(\mathcal{L}) = \frac{1}{|\mathcal{L}|} \sum_{(\boldsymbol{x},y)\in\mathcal{L}} [\![y = c]\!]$$

where $[\![y = c]\!] = 1$ if $y = c$ and 0 otherwise

Leaf Purity

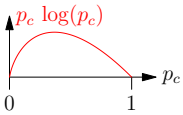
- Two different purity measures, $Q_m(\mathcal{L})$, for a leaf node $\mathcal{L}$ are commonly used
  - **Gini index**

$$Q_m^g(\mathcal{L}) = \sum_{c\in\mathcal{C}} p_c(\mathcal{L})\ (1 - p_c(\mathcal{L}))$$

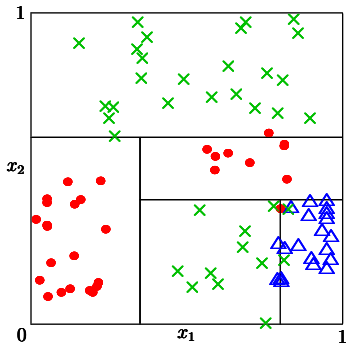


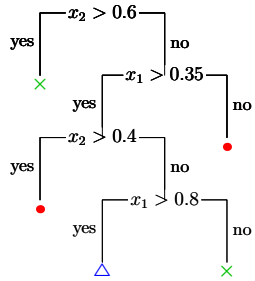
  - **Cross-entropy**

$$Q_m^e(\mathcal{L}) = -\sum_{c\in\mathcal{C}} p_c(\mathcal{L})\ \log(p_c(\mathcal{L}))$$



Building Decision Trees



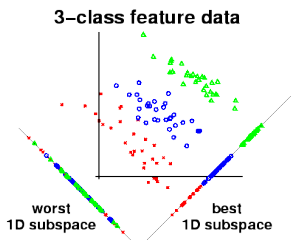


Observations

- Decision trees are very useful for exploring new data sets—the tree shows what features are most important
- That is, features near the root of the tree are most informative
- CART is a classic implementation that builds **C**lassification **A**nd **R**egression **T**rees
- Decision trees handle missing data well
- A single decision is usually not that good, but an ensemble of decision trees (random forest or boosting) can be very good

Outline

1. Classification
2. Finding Relevant Variables
3. **Feature Selection**
4. Clustering



Feature Selection

- In biology many features may be irrelevant to classification
- You could try every combination of feature, but for p features there are 2^p combinations
- There are various schemes for trying to find which features are important

Correlation with Class

- You can measure the correlation between each feature and the class category (assuming binary classification $y_i \in \{-1, 1\}$)

$$r = \frac{\mathbb{E}\left[(x_i - \mathbb{E}[x_i])(y_i - \mathbb{E}[y_i])\right]}{\sqrt{\mathbb{V}\mathrm{ar}[x_i]\ \mathbb{V}\mathrm{ar}[y_i]}}$$

- But be aware
  - If two variables are correlated with each other both may not provide any more information than one alone
  - Two variables that aren't correlated with the data, might together predict classes perfectly

Building Classifiers

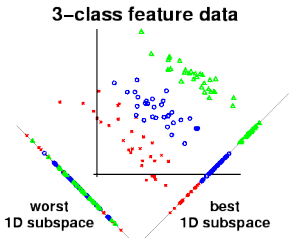
- We can systematically build a classifier starting from the most informative variable
- We then add all possible variables and choose the best to form a two variable classifier
- This can be repeated until we find that adding additional variables doesn't improve generalisation performance
- We can also start with all variables and eliminate the worst variables one by one

Feature Selection

- There are whole books written on feature selection
- Techniques exist, such as Lasso (adding an L_1 regulariser), that are designed to do automatic feature selection
- Often in biology we want to know if we can make reasonably reliable predictions with just a few variables
- Don't necessarily need very sophisticated techniques to do this

Outline

1. Classification
2. Finding Relevant Variables
3. Feature Selection
4. **Clustering**



Clustering

- In determining whether a cell is cancerous it is often useful to find projections of the data where clustering is more obvious
- A projection is a set of ortho-normal vectors

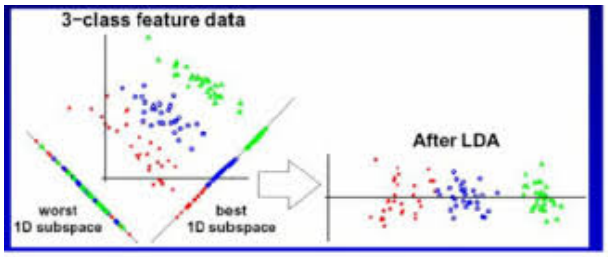
$$\mathbf{P} = (\boldsymbol{p}_1 \quad \boldsymbol{p}_2 \quad \cdots \quad \boldsymbol{p}_k)$$

such that we can project a p-dimensional vector, $\boldsymbol{x}$, to a k-dimensional vector, $\boldsymbol{z} = \mathbf{P}^{\mathsf{T}}\boldsymbol{x}$

Fisher Discriminant Analysis & LDA

- If we want to find the best 1-d projection that maximises the ratio of inter-class distance to intra-class distances then we use Fisher Discriminant Analysis
- We can generalise this to the best k-d projection using *linear discriminant analysis* or LDA
- We can use LDA for multiple classes
- There are standard packages for running LDA (beware not to confuse linear discriminant analysis with latent Dirichlet allocation)

Linear Discrimant Analysis



Visualisation

- Using LDA in 2 or 3 dimensions often allows you to visualise the clustering of data
- This can give an idea of what cancers are like each other
- LDA mixes different variables so doesn't help with feature selection
- Other techniques such as **t-SNE** are also useful for visualising cluster

Conclusions

- We've covered some techniques for understand data
- There are many more you could try
- But you don't need to try everything
- Write programs (scripts) for doing data analysis as you will need the over and over again
- Visualise as much as possible
- Python can be very good to do this